Y1 Summer Project Manual

Haris Forde, Vincent Leung, Dhruv Mathur, Archie Lentzos

Version 0.1

As of 26/3/2026

Contents

0	Introduction	**3**
I	**Physics**	
1	Curved Spacetime	**5**
2	Observer in Motion	**8**
3	Light in Media	**11**
4	Colours	**13**
II	**Algorithms**	
5	Numerical Integration	**16**
6	Hermite Interpolation	**19**
III	**Code**	
7	Classes	**22**

Chapter 0

Introduction

This is a *fully computational* project that aims to investigate the optical phenomena produced by black holes as well as the physical motion and properties of its accretion disc with which it is commonly observed in tandem in astronomy. The project's primary objective is to produce a realistic and physically accurate picture/video of a black hole and its accretion disc through computer simulations, and to quantify the most important results obtained from its outputs. As of writing this document, the latter is largely undetermined.

This manual serves as a guide to the numerous components of the project, from a general crash course on the physics and principles behind the simulation to the specific mechanisms of the code employed. The code for the project is uploaded into a GitHub repository, which may be found by clicking on this link.

Throughout much of the project, we will work in *natural units*. There are different conventions, but here we choose $G = c = h = k = 1$ and $\varepsilon_0 = 1/4\pi$. This reduces a significant bulk of the expressions we work with, and the factors are completely recoverable by considering the dimensions of the relevant quantities. We also work with the sign convention $(-, +, +, +)$.

The developer may also do well to note that as important as computational efficiency is in this project, physical accuracy of the simulation remains an important priority. Approximations are *not* made unless necessary or with convincing reason. The physics employed should be kept general enough to allow for extensions to different or more complicated scenarios without much re-derivation, and the code should be structured enough to be readable and user-friendly to a reasonable extent. Additionally, to avoid re-editing the manual every time new code is committed, only include the relevant architecture of the code that will likely be unchanged or unaltered significantly, at least for quite a while.

Manual written by Vincent Leung.

Part I

Physics

Chapter 1

Curved Spacetime

In general relativity, gravity is described not by a force, but by a *curvature* in spacetime. All geometric information of the spacetime manifold is captured in the **metric tensor**, a rank-(0,2) tensor commonly denoted $g_{\mu\nu}$. Intuitively, it describes the form of a line element as

$$ds^2 = g_{\mu\nu}\, dx^\mu\, dx^\nu, \tag{1.1}$$

with summation over every pair of Greek indices applied. In a region with no energy, spacetime is described by the **Minkowski metric**, usually denoted $\eta_{\mu\nu} \sim \mathrm{diag}\,(-1, 1, 1, 1)$ in **Minkowski coordinates** (t, x, y, z), essentially the 4D-analogue to Cartesian coordinates. One can plug this definition into Eq. (1.1) to recover the formula for the spacetime interval.

In general, given a physical system in which energy moves within spacetime, the metric tensor is obtained by solving the **Einstein's field equations** (EFE). We will not focus on doing so here. For all purposes we are concerned with, the relevant metric is already well-known:

Definition 1.1. The **Schwarzschild metric**, which describes the metric tensor of a neutral, non-rotating black hole of mass M. In **Schwarzschild coordinates** (t, r, θ, ϕ) (essentially spherical coordinates with an additional temporal dimension), it is given by:

$$ds^2 = -\left(1 - \frac{2M}{r}\right)\, dt^2 + \left(1 - \frac{2M}{r}\right)^{-1}\, dr^2 + r^2\, d\theta^2 + r^2 \sin^2\theta\, d\phi^2 \tag{1.2}$$

One may check that this reduces to the Minkowsi metric when $r \to \infty$.

Definition 1.2. The **Kerr metric**, which describes the metric tensor of a neutral but rotating black hole of mass M and angular momentum J. In **Boyer-Lindquist coordinates** (t, r, θ, ϕ) with parameter $a = J/M$, it is given by:

$$\begin{aligned} ds^2 = &-\left(1 - \frac{2Mr}{\Sigma}\right)\, dt^2 - \frac{4Mra\sin^2\theta}{\Sigma}\, dt d\phi + \frac{\Sigma}{\Delta}\, dr^2 \\ &+ \Sigma\, d\theta^2 + \left(r^2 + a^2 + \frac{2Mra^2\sin^2\theta}{\Sigma}\right)\sin^2\theta\, d\phi^2 \end{aligned} \tag{1.3}$$

where $\Sigma = r^2 + a^2\cos^2\theta$ and $\Delta = r^2 - 2Mr + a^2$. The coordinate transformations from Boyer-Lindquist to Minkowski are given by:

$$\begin{cases} x = \sqrt{r^2 + a^2}\sin\theta\cos\phi \\ y = \sqrt{r^2 + a^2}\sin\theta\sin\phi \\ z = r\cos\theta \end{cases} \tag{1.4}$$

This set of equations may be inverted to obtain the transformation going the other way.

Definition 1.3. The **Kerr-Newman metric**, which describes the most general black hole of mass M, angular momentum J, and charge Q. In Boyer-Lindquist coordinates (t, r, θ, ϕ) with parameter $a = J/M$, it is given by:

$$\begin{aligned} ds^2 = &-\left(1 - \frac{2Mr - Q^2}{\Sigma}\right) dt^2 - \frac{4Mra\sin^2\theta}{\Sigma}\, dt d\phi + \frac{\Sigma}{\Delta}\, dr^2 \\ &+ \Sigma\, d\theta^2 + \left[r^2 + a^2 + \frac{(2Mr - Q^2)\, a^2 \sin^2\theta}{\Sigma}\right] \sin^2\theta\, d\phi^2 \end{aligned} \tag{1.5}$$

where $\Sigma = r^2 + a^2\cos^2\theta$, and $\Delta = r^2 - 2Mr + Q^2 + a^2$.

One important thing to note is that given a black hole mass M, the other parameters cannot simply take on any value. In particular, the inequality

$$\left(\frac{J}{M}\right)^2 + Q^2 \leq M^2 \tag{1.6}$$

must be satisfied. Such black holes are termed *sub-extremal*, or if the relation is an equality, *extremal*. Such black holes have a singularity hidden behind its event horizon. In contrast, *super-extremal* black holes which violate this inequality exhibit **naked singularities**, which violates the **cosmic censorship theorem**. Many physicists believe that they cannot exist, so we will simply ignore these combination of parameters as unphysical.

The important thing about the metric tensor is that it tells us how objects move in spacetime. Objects in free-fall follow what is known as a **geodesic**, a path obtained by transporting a vector parallel to its direction.

Theorem 1.4. Suppose the worldline of an object is parametrised by a variable λ such that its location within spacetime is described by $x^\mu(\lambda)$. Then the following equation is satisfied:

$$\frac{d^2x^\mu}{d\lambda^2} = -\Gamma^\mu_{\alpha\beta}\frac{dx^\alpha}{d\lambda}\frac{dx^\beta}{d\lambda}, \tag{1.7}$$

where $\Gamma^\gamma_{\alpha\beta}$ are the **Christoffel symbols** of the second kind, given by:

$$\Gamma^\gamma_{\alpha\beta} = \frac{g^{\gamma\sigma}}{2}\left(\frac{\partial g_{\sigma\alpha}}{\partial x^\beta} + \frac{\partial g_{\sigma\beta}}{\partial x^\alpha} - \frac{\partial g_{\alpha\beta}}{\partial x^\sigma}\right), \tag{1.8}$$

which are symmetric in its two lower indices. Here, $g^{\mu\nu}$ is the **inverse metric**. Eq. (1.7) is called the **geodesic equation**.

The geodesic equation Eq. (1.7) is the core of the ray-tracing (or rather, ray-marching, as we will explain later) process. Unlike in Newtonian gravity, light is *curved* by the presence of mass since it, too, follows a geodesic in *curved* spacetime. Thus, if we are given an initial position and initial direction, we can simply perform numerical integration with (1.7) to obtain the path of the light ray.

There is one more important caveat – in reality, we only specify the four-position (t, x, y, z) (when and where the light ray originates) and the normalised wave *three-vector* (k^x, k^y, k^z) (its direction in space) of the light ray. We are still missing the temporal component of its velocity to complete the set of initial values.

Theorem 1.5. The geodesic of a light ray connects two events separated by a light-like interval,

i.e. $ds^2 = 0$. Using Eq. (1.1), one may write

$$\begin{aligned} ds^2 &= g_{\mu\nu}\, dx^\mu\, dx^\nu = 0 \\ \therefore g_{\mu\nu} \frac{dx^\mu}{d\lambda} \frac{dx^\nu}{d\lambda} &= 0 \end{aligned} \tag{1.9}$$

This is called the **null condition**, which may be solved for to obtain $k^t = dt/d\lambda$. This equation is satisfied everywhere along the geodesic of a light ray.

Since the metric in Eq. (1.9) is given in metric coordinates, in practice we will need to perform a coordinate transformation on the wave four-vector (k^t, k^x, k^y, k^z) using the Jacobian before plugging it into Eq. (1.9). Nevertheless, the equation is always quadratic in v^t, so computing the solution for k^t is not difficult.

Also, since we will be doing ray-*marching*, where we trace geodesics from the camera to a position within the physical scene, our 'velocity' vector should have a negative v^t component because we are tracing *backwards* in time. This is important to keep in mind when we solve the null condition.

Summary. One can trace the path of light rays within a spacetime described by an arbitrary metric as follows: given a position within spacetime and a three-vector in Minkowski coordinates with which the a light ray originates from and in the direction of, one may solve for its temporal component using the null condition Eq. (1.9) to complete the set of initial values for numerical integration of Eq. (1.7). Since the ray-marching technique is employed, k^t is negative, and the integration continues until a solid object is hit, or if it falls into the black hole, or escapes into the background.

Chapter 2

Observer in Motion

In this project, we also hope to let the camera move to produce a more immersive and realistic rendering. The machinery required to predict its motion has all been developed in Chapter 1, with a slight modification that the right hand side of the 'null condition' becomes not 0, but -1, because the geodesic connects spacetime events separated by a *time-like* interval, and that this time the *positive* solution when solving the null condition is taken, because we are tracing the path of the camera as it evolves forwards in time. Also, we take the parameter λ to be the proper time of the camera τ, which allows for a physically accurate passage of time in the final rendered product.

Theorem 2.1. The worldline of a massive object obeys the geodesic equation (1.7) as well as:

$$\frac{d^2x^\mu}{d\tau^2} = -\Gamma^\mu_{\alpha\beta}\frac{dx^\alpha}{d\tau}\frac{dx^\beta}{d\tau}, \quad \text{and} \quad g_{\mu\nu}\frac{dx^\mu}{d\tau}\frac{dx^\nu}{d\tau} = -1, \tag{2.1}$$

the latter of which may be solved for to obtain $v^t = dt/d\tau$, and the former numerically integrated to yield the worldline of the object.

But with this comes another problem: as of now, we have only treated light as a ray, neglecting its wave-like properties. When we let the camera move, wave phenomena come into play and make a significant difference on the final image. However, if we were to simulate these too as emergent phenomena of more fundamental principles, we would have to simulate the full electromagnetic field within which it propagates – this would be way too much computational work for the phenomena relevant to the rendering. Instead, we choose to hard-code these effects into the rendering engine.

It is common knowledge that the perceived frequency of a wave emitted from a moving source is not the same as that of the source itself, and the same goes for a moving observer – this is the classical **Doppler effect**. However, the classical formula defines an absolute frame in which the velocities are defined, i.e. that of the medium through which the wave propagates. By the principle of relativity, this cannot be the case; any valid formula must be **Lorentz invariant**, i.e. the expression stays the same under a Lorentz transformation.

Moreover, consider the effects of gravity. If a light ray travels within a gravitational field to an observer, by the **equivalence principle** the perceived light must be the same as if the observer itself is accelerating. And we know *a priori* that the perceived frequency changes due to the Doppler effect, even if the observer, relative to a frame at infinity, is stationary. This phenomenon is known as **gravitational redshift**.

Theorem 2.2. Suppose a light ray is emitted from a location within spacetime from a source moving at proper velocity $(u^\mu)_{\text{src}}$ and ends at an observer moving at proper velocity $(u^\mu)_{\text{obs}}$. Then

the observed and source frequencies are related by:

$$\frac{f_{\text{obs}}}{f_{\text{src}}} = D := \frac{\left(g_{\mu\nu}k^{\mu}u^{\nu}\right)_{\text{obs}}}{\left(g_{\mu\nu}k^{\mu}u^{\nu}\right)_{\text{src}}}, \tag{2.2}$$

where k^{μ} is the wave four-vector of the light ray, and the subscripts indicate where the quantities are evaluated at. Here, D is known as the **Doppler factor**. This equation incorporates both Doppler shift *and* gravitational redshift/blueshift.

In practice, the accretion disc, which is the only light source in the scene, would emit light over a range of frequencies. Fret not, Eq. (2.2) applies to every single one, and the frequency range may simply be multiplied by D.

In addition to changes in frequency, at relativistic speeds the *intensity* of the light also changes. In the special relativistic case, one may realize this simply by performing a Lorentz transformation on the **electromagnetic field tensor** $F^{\mu\nu}$ describing an EM wave – the amplitude decreases by a factor of D. The intensity is proportional to the square of the amplitude, so it decreases by a factor of D^2.

For simplicity, let's suppose for now that the accretion disc is a blackbody, and so emits light over a range of frequencies, i.e. a spectrum. We talk of **spectral radiance**, which has units of intensity per unit solid angle per unit frequency. It's analogous to a probability density – the area under the spectral curve gives the intensity of the emitted light within the specified range of frequencies.

Theorem 2.3. The spectral radiance of a blackbody in thermal equilibrium at temperature T is

$$B\left(f\right) = \frac{2f^3}{e^{f/T} - 1}. \tag{2.3}$$

This is known as **Planck's law**. The intensity of the emitted light within a specified range of frequencies between f and $f + \Delta f$ is then given by:

$$I = \int_{f}^{f+\Delta f} B\left(f'\right)\ df'. \tag{2.4}$$

Of course, Eq. (2.3) is in the frame of an observer at infinity. When we perform a Lorentz transformation to a moving frame, the frequency is multiplied by D, the intensity is divided by D^2, and so simple dimensional analysis reveals that the spectral radiance is divided by D^3. The *observed* spectral radiance is then

$$B_{\text{obs}}\left(f\right) = \frac{B\left(Df\right)}{D^3} = \frac{2f^3}{e^{Df/T} - 1}. \tag{2.5}$$

Curiously, this is precisely the spectrum of a blackbody with temperature T/D. Further processing of the spectrum should then be able to give the actual colour observed by the camera due to the relativistic Doppler effect.

Before we move on, there is also one important optical phenomenon to note for moving observers: **relativistic aberration**. This is not a consequence of the wave-like nature of light, but a simple relativistic effect. We perform our ray-marching by 'shooting' light rays (backwards in time) into the viewport, but we must also consider the motion of the camera itself – the velocities 'add' in a way such that the direction of the light as seen from an outside observer is not the same as in the frame of the camera.

Theorem 2.4. Suppose the camera moves in the positive x-direction with velocity β relative to a stationary outside observer at infinity, and that it receives a light ray at an angle θ to the positive

x-axis *in its own frame*. The angle at which the light ray makes with the camera's direction of motion relative to the outside observer, θ', is given by:

$$\tan\theta' = \frac{\sin\theta}{\gamma\left(\cos\theta + \beta\right)}. \tag{2.6}$$

This is easily derivable from Einstein's **velocity-addition formula**.

In practice, then, when we want to 'shoot' a light ray with some specified direction in the frame of the camera, we solve for θ' using Eq. (2.6), and in the plane defined by the direction vector of the ray and the camera's velocity vector, alter the direction of the light ray accordingly, before continuing on to numerical integration.

There is also another consequence of relativistic aberration – if light rays reach the observer uniformly in all directions in the frame of an outside observer, Eq. (2.6) says that in the frame of the camera, the light rays would be more concentrated in the front, and sparser at the back. This leads to a variation in the *intensity per unit solid angle*, which alters the spectral radiance received. The effect of this is that objects would appear brighter in front, and darker at the back.

In particular, if $d\Omega$ is a patch of solid angle in the camera's sky through which light rays reach the camera, then in the frame of an outside observer, the same light rays would reach the camera over a solid angle $d\Omega'$, which is related to $d\Omega$ through Eq. (2.6):

$$\frac{d\Omega'}{d\Omega} = \frac{\sin\theta'}{\sin\theta}\frac{d\theta'}{d\theta} = \frac{\sin\theta'}{\sin\theta}\frac{\gamma\left(1+\beta\cos\theta\right)}{\sin^2\theta + \gamma^2\left(\cos\theta+\beta\right)^2}. \tag{2.7}$$

In the following sections we will consider spectral radiances relative to an outside observer. We should therefore *divide* by the factor in Eq. (2.7). As a sanity check, a sufficiently small θ would make the factor smaller than 1, so dividing by it would increase the spectral radiance and therefore the perceived brightness, and the converse is also true, as expected. Also, if we set $\beta = 0$ then the factor is simply 1 everywhere – no relativistic aberration. And as we increase β the effects of this exaggerate dramatically too. Eq. (2.7) seems sensible.

Summary. For an observer in motion, we first predict its motion through spacetime using Eq. (2.1). Next, to consider the effects of relativistic aberration, for any specified direction of light ray in the frame of the camera, we use Eq. (2.6) to solve for its spatial direction in the frame of an outside observer at infinity before integrating the geodesic. Since light is also a wave, we must also consider the relativistic Doppler effect on the observed spectral radiance (if any) using Eq. (2.5). The final perceived spectral radiance should also be divided by the factor in Eq. (2.7) to account for aberration.

Chapter 3

Light in Media

In this simulation there will also be gases, i.e. the accretion disc itself. Since it is charged, it is not inconceivable that it will also have a non-negligible refractive index that will alter how the light moves. Thus, in addition to the effects of curved spacetime, we must also take into account the medium in which light travels through.

To good approximation we can assume that the medium is linear, isotropic, and non-dispersive, and that it is fully characterised by two electromagnetic variables: the permittivity ε and permeability μ.

Theorem 3.1. Within a linear medium of permittivity ε and permeability μ, light follows the geodesic corresponding to the metric

$$\hat{g}_{\mu\nu} = g_{\mu\nu} + \left(1 - \frac{1}{\varepsilon\mu}\right) u_\mu u_\nu, \tag{3.1}$$

where $g_{\mu\nu}$ is the actual metric (in this case the one from the black hole), and u^μ is the proper velocity of the medium. Note that Eq. (3.1) uses the covariant version. Here, $\hat{g}_{\mu\nu}$ is called the **optical metric**.

As we will later explain, the gas can be thought of as a fluid that permeates the entirety of the scene. In this most general case, it is simplest to separate the optical metric from the physical metric, the former of which is used to trace light geodesics, and the latter for the motion of the camera along with the accretion disc. Unfortunately, there is no simple formula that relates the old Christoffel symbols to the new one – we would have to re-derive them. This is hardly an issue, since there wouldn't be an analytic expression describing the optical metric anyway, and we will be forced to use numerical approximations.

The gas does not only alter the *path* of the light ray, but also its intensity. Molecules within the gas can absorb and scatter incoming light with different mechanisms, e.g. Thomson, Compton, etc. In any case, these effects can be captured in two quantities called the **absorption coefficient** α and the **scattering coefficient** σ, which may then be added to yield the **extinction coefficient** κ that directly determines the attenuation of the light ray. This quantity varies with space and time depending on the properties of the fluid, so the decrease in intensity is specific to the path traced out by the light. The developer may also do well to keep in mind that $\alpha, \sigma, \kappa \to 0$, and $n = \sqrt{\varepsilon_r \mu_r} \to 1$ as the density of the gas vanishes, i.e. $\rho \to 0$.

Theorem 3.2. Suppose a light ray enters a medium of absorption and scattering coefficients α and σ. Let I be the spectral radiance of the light. Then

$$\frac{dI}{d\lambda} = -\kappa I + \alpha B_{\text{obs}} + \sigma J_{\text{obs}} \tag{3.2}$$

where $\kappa = \alpha + \sigma$ is the extinction coefficient, B is the spectral radiance of a blackbody corresponding to the local temperature, and J is the **effective scattering spectral radiance**, an effective source that captures the additional intensity due to scattering. An equation of this form is known as a **radiative transfer equation** (RTE).

Eq. (3.2) says that the change in the intensity along the light's optical path is due to a loss term proportional to intensity, and two 'gain' terms, one due to blackbody radiation, and the other incoming scattering. The former is obtained from Planck's law Eq. (2.3), then applying the effects of gravitational redshift with Eq. (2.5). The latter is given as a function over the domain of the fluid. It may be given as some kind of analytic expression, or precomputed before the rendering. Either way, it, too, should also be Doppler-shifted before plugging into the equation Eq. (3.2).

A good starting point for J is to assume that the accretion disc is thin, so that it radiates roughly like a blackbody on each side. If we also take J to be the mean spectral radiance of the disc, then there exists a formula for J that is, although approximate, analytic.

Theorem 3.3. In a thin disk, the effective scattering spectral radiance may be taken to be the spectral radiance of a blackbody of some effective temperature T_{eff}:

$$J(f) = \frac{2f^3}{e^{f/T_{\text{eff}}} - 1}. \tag{3.3}$$

Here, the effective temperature is given by the **Shakura-Sunyaev formula**:

$$\sigma T_{\text{eff}}^4 = \frac{3M\dot{M}}{8\pi r^3}\left(1 - \sqrt{\frac{r_{\text{in}}}{r}}\right), \tag{3.4}$$

where $\sigma = 2\pi^5/15$ is the Stefan-Boltzmann constant, M the mass of the black hole, $\dot{M}$ the accretion rate, r_{in} the inner radius of the disc, and r the cylindrical distance from the centre of the black hole.

It is tempting to solve the differential equation Eq. (3.2) simultaneously with the geodesic equation Eq. (1.7) to maximize computational efficiency, but this is unfortunately not possible because the direction of integration is simply not the same (solving for I won't help either, since the solution features two *separate* integrations). One must start at the source and integrate towards the observer to obtain the final spectral radiance observed at the camera.

The reader may start to wonder how exactly we can store the entirety of the spectral radiance as a *function* of frequency. The answer is that we store it as a large-dimensional vector, with each entry being the spectral radiance evaluated over a fixed interval (say 5 nm). Integration of the graph (see Chapter 4) then becomes trivial – it is simply the sum of the entries multiplied by the interval.

One may preemptively think that, since the range of human color vision is only within a certain frequency range, it will do no harm to only store the spectral radiance values over said range. This is not true because of the Doppler effect. According to Eq. (2.5), we would need to refer to values at least in the human color vision range *multiplied* by the Doppler factor D, which is an unknown until the integration is actually performed.

Summary. Light within a linear medium of permittivity ε and permeability μ follows a geodesic of the *optical* metric as given in Eq. (3.1). The spectral radiance of the light ray can be obtained by performing numerical integration of the RTE Eq. (3.2) given the absorption and scattering coefficients of the gas α and σ, as well as the effective scattering spectral radiance J.

Chapter 4

Colours

There remains the question of how we can convert intensities and frequencies back into a colour on a .png/.jpg image. In this project, we use a well-established procedure that turns a spectral radiance profile into an RGB colour ready to be displayed digitally on a computer.

Definition 4.1. Suppose one is given a spectral radiance graph in terms of *wavelength* $I(\lambda)$. The **CIE 1931 colorimetric observer** models human colour-vision with three functions:

$$\begin{cases} \bar{x}(\lambda) = 1.056g(\lambda; 599.8, 0.0264, 0.0323) + 0.362g(\lambda; 442.0, 0.0624, 0.0374) \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad - 0.065g(\lambda; 501.1, 0.0490, 0.0382), \\ \bar{y}(\lambda) = 0.821g(\lambda; 568.8, 0.0213, 0.0247) + 0.286g(\lambda; 530.9, 0.0613, 0.0322), \\ \bar{z}(\lambda) = 1.217g(\lambda; 437.0, 0.0845, 0.0278) + 0.681g(\lambda; 459.0, 0.0385, 0.0725), \end{cases} \tag{4.1}$$

where the arguments of g are given in nanometres, defined by:

$$g(\lambda; \mu, \tau_1, \tau_2) = \begin{cases} \exp\left[-\tau_1^2 (x-\mu)^2/2\right], & x < \mu, \\ \exp\left[-\tau_2^2 (x-\mu)^2/2\right], & x \geq \mu. \end{cases} \tag{4.2}$$

One may calculate the inner product of these functions with the spectral radiance $I(\lambda)$ to obtain what is known as the **tristimulus values**:

$$X = \int_{380}^{780} I(\lambda)\,\bar{x}(\lambda)\ d\lambda, \quad Y = \int_{380}^{780} I(\lambda)\,\bar{y}(\lambda)\ d\lambda, \quad Z = \int_{380}^{780} I(\lambda)\,\bar{z}(\lambda)\ d\lambda. \tag{4.3}$$

These values can then be used to obtain the sRGB values for display:

$$\begin{pmatrix} R \\ G \\ B \end{pmatrix} = \begin{pmatrix} 3.2406 & -1.5372 & -0.4986 \\ -0.9689 & 1.8758 & 0.0415 \\ 0.0557 & -0.2040 & 1.0570 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \tag{4.4}$$

Multiply through by 255 to obtain the final result.

This is by no means a short process – the spectral radiance graph must first be converted into wavelength: $I(\lambda) = I(1/f)$, and then into SI units for the integration in Eq. (4.3) (or convert the bounds into natural units). Luckily, if we treat the spectral radiance graph as a vector, this becomes simply a matrix product – and this only needs to be done *once* per ray, and can even be delayed to post-processing.

Sometimes it might help to do the opposite – from a colour, determine a spectral radiance graph. This may be useful when we want to render an image that includes a stock image as a background, or if we want to have an object emitting light at a specified colour – then, all the machinery of

Chapters 2 and 3 may be utilised to obtain the observed colour modified by the other objects in the scene. The forward direction is specified by Definition 4.1, but the backwards one is not unique; an infinity of spectral radiance graphs can yield the same colour to the human eye.

Here we reconstruct the smoothest possible spectral curve using a method developed by Scott A. Burns. The idea is to minimise the squared slope of the spectral radiance curve, while asserting that the curve must integrate to give the known tristimulus values (obtained by inverting Eq. (4.4)). This then becomes an **optimisation problem**. If we represent the spectral radiance as a vector as described in Chapter 3, this is a system of linear equations, which can be easily solved.

Theorem 4.2. Given a colour with tristimulus values X, Y, and Z, to compute the corresponding vector $\mathbf{I}$ (of N dimensions) describing the smoothest spectral radiance, compute the $(N+3) \times (N+3)$ matrix:

$$B = \begin{pmatrix} D & A_W \\ A_W^T & 0 \end{pmatrix}, \tag{4.5}$$

where D is an $N \times N$ tridiagonal matrix, and A_W an $N \times 3$ matrix. They are given by:

$$D = \begin{pmatrix} 2 & -2 & & & & \\ -2 & 4 & -2 & & & \\ & -2 & 4 & -2 & & \\ & & \ddots & \ddots & \ddots & \\ & & & -2 & 4 & -2 \\ & & & & -2 & 2 \end{pmatrix}, \quad A_W = \begin{pmatrix} \bar{x}(\lambda_1) & \bar{y}(\lambda_1) & \bar{z}(\lambda_1) \\ \bar{x}(\lambda_2) & \bar{y}(\lambda_2) & \bar{z}(\lambda_2) \\ \vdots & \vdots & \vdots \\ \bar{x}(\lambda_N) & \bar{y}(\lambda_N) & \bar{z}(\lambda_N) \end{pmatrix}, \tag{4.6}$$

where the λ_i's are the wavelengths at which the spectral radiance is evaluated at in the desired vector $\mathbf{I}$. Now, this vector is given by the first N entries of:

$$\mathbf{I}' = B^{-1} \begin{pmatrix} 0 & 0 & \cdots & 0 & X & Y & Z \end{pmatrix}^T. \tag{4.7}$$

This is known as the **smoothest reflectance reconstruction** method.

The computational workload here is mainly solving for the inverse of the matrix B, which may contain a large number of dimensions. There is a saving grace in that we can simply restrict our wavelength domain to the human colour range – 380-780 nm as used in Eq. (4.3) – and pick a suitable interval to work with from there. For example, if $\Delta\lambda = 5$ nm, then $N = 81$, and the computer is going to have to solve for the inverse of an 84×84 matrix. The nice thing about this is that B is dependent only on the choice of λ_i's, and so its inverse only has to be computed once before the rendering to convert every colour under the sun into a spectral graph.

Summary. To convert a spectral radiance graph to a colour, use the procedure of Definition 4.1. There are infinitely many spectral radiances that produce a given colour, but we would like to pick the smoothest looking curve. The procedure that does exactly that, the smoothest reflection reconstruction method, is given in Theorem 4.2.

Part II

Algorithms

Chapter 5

Numerical Integration

This section focuses on how numerical integration, specifically of the geodesic equation Eq. (1.7), is done. This is an **initial value problem**, i.e. we are given the initial value of the function we wish to find the form of at later parameter values.

To solve a differential equation, the basic idea is to start at the initial value, take some small step h forwards, and compute the new value of the function based on its derivative, whose value at that point is given by the differential equation. There are many ways of computing this, here we use the **Runge-Kutta method** (RK4).

Definition 5.1. Suppose we are given a differential equation

$$\frac{dy}{dx} = f(x, y). \tag{5.1}$$

The **RK4 method** states that, if $y_n := y(x_n)$ is given, then the approximation of $y(x_{n+1}) := y(x + h)$, for some h, denoted y_{n+1}, is given by

$$y_{n+1} = y_n + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right), \tag{5.2}$$

where

$$\begin{cases} k_1 = f(x_n, y_n), \\ k_2 = f(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_1), \\ k_3 = f(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_2), \\ k_4 = f(x_n + h, y_n + hk_3) \end{cases}. \tag{5.3}$$

The smaller h is, the closer y_{n+1} is to $y(x_{n+1})$. This defines a recursive method to approximate $y(x)$ for any $x > x_0$ given some initial value $y(x_0) = y_0$.

At first glance, the geodesic equation Eq. (1.7) might not apply because it is a *second*-order differential equation – but note that we can rewrite it as

$$\frac{d\mathbf{y}}{d\lambda} = \mathbf{f}(\mathbf{y}) = \begin{pmatrix} k^\mu \\ -\Gamma^\mu_{\alpha\beta} k^\alpha k^\beta \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} x^\mu \\ k^\mu \end{pmatrix}, \tag{5.4}$$

where $k^\mu := dx^\mu/d\lambda$. Effectively, $\mathbf{y}$ is an 8-dimensional vector, and the RK4 method applies to its each and every entry.

There is still a question of what step size h we should pick. In regions where dy/dx changes quickly, the step size should be small, because too large of an h would produce too big of an error. Conversely, if dy/dx stays approximately constant, too small of a step size would, although produce a very accurate step, be very computationally inefficient. This means that h must be able to adapt

to an estimation of the **error** between the computed and actual value.

How do we estimate an error? We use the **Richardson extrapolation**, which assumes that the error may be written as a Taylor series in h. The RK4 method has an error on the order of $O(h^5)$ for each step, so the leading term is on the order of h^5. We can then write something like

$$y(x_{n+1}) = y_n + Ch^5 + O(h^6), \tag{5.5}$$

and the **error** of the approximation would be approximately Ch^5.

To actually solve for the error term, we employ a trick called **step-doubling**, where we take two smaller 'fine' steps $h/2$ and apply RK4 twice, in addition to the big 'coarse' step of size h. The resulting error term in y_{n+1}, after repeating the same argument as before, is $C(h/2)^5 \cdot 2 = Ch^5/16$. Since we have already computed a *finer* solution, we might as well take $\epsilon := Ch^5/16$ to be our error term. We can then solve for this pretty easily:

$$\begin{aligned} y_{\text{crs}} - y_{\text{fin}} &= \frac{15}{16}Ch^5 \\ \therefore \epsilon = \frac{Ch^5}{16} &= \frac{y_{\text{crs}} - y_{\text{fin}}}{15}. \end{aligned} \tag{5.6}$$

Eq. (5.6) gives a formula for the error of the RK4 approximation. The magnitude of this error will depend on the rate of change of dy/dx – if it is too big then the approximation might become too inaccurate. Based on the value of ϵ we can choose to accept or discard the solution y_{fin}, by checking if it is below a certain **tolerance** τ. If it is, then we accept the solution and be a bit bold by increasing the step size; if it isn't, then we shall reject it and be a bit conservative by decreasing the step size.

There is some wisdom in choosing the tolerance τ. Too big and the integration would be inaccurate, too small and it would take too long. A good value after playing around with the code a bit is $\tau = 10^{-8}$. In any case, τ dictates the *size* of the error per integration step. If we adapt the step size right, in the ideal case, the error in the approximation should be around the value of τ.

To determine the factor by which we scale the step size, we simply take a look at Eq. (5.5), and conclude that $h \propto \epsilon^{1/5}$. If ideally we want our error to be the tolerance τ, then we can write:

$$\frac{h_{\text{new}}}{h} = \left(\frac{\tau}{\epsilon}\right)^{1/5}, \tag{5.7}$$

i.e. we multiply h by $(\tau/\epsilon)^{1/5}$. If we want to be a bit more conservative, we can insert some **safety factor** in front, say, 0.9. We can also set some bounds on how big or small we will allow the integrator to scale its steps – it might not be a good idea to scale h by a number too large like 10 or one too small like 0.01 even if the factor in Eq. (5.7) is so. In our code we set the bounds at 0.1 and 5.

Either way, Eq. (5.7) is used to obtain a new step size, whether or not if the step is accepted or rejected. Over a couple iterations the step size would adapt to the desired tolerance, and we may integrate away to our heart's content until x reaches some maximum value x_{max} or if we want to terminate the integration early. In our case, the light ray may hit some solid object, or the black hole's singularity. The latter is easy to detect – check if the step size becomes too small (say, below 10^{-4}) and if the metric blows up (say, above 200). The former introduces a bit of a problem, which we shall continue to discuss.

The accretion disc is a *thin* region. So thin, in fact, that the integrator may simply skip over the disc entirely because the integration is going so well and h is at some healthy not-too-small value. To actually make it *detect* the thin region, we have to cap h based on the distance to the nearest object within the scene. Mathematically, and generalising to a *vectorial* solution $\mathbf{y}$ (called the **state vector**) as in Eq. (5.4), if the state vector is approaching some region R in state space

we're interested in, we want to cap its evolution step h so it doesn't overshoot the region – it must be smaller than the minimum distance D to R, divided by its 'speed' in state space, i.e.

$$h \leq \frac{D}{\|d\mathbf{y}/dx\|}. \tag{5.8}$$

If we were to implement Eq. (5.8) directly, $\mathbf{y}$ would approach R asymptotically – at best it will get stuck on the boundary of R. So we introduce some factor larger than 1 in front, say 1.25, so that it actually steps *into* R and triggers detection accordingly. Now, introducing this factor would indeed make it possible to overshoot R (which would undermine the entire point of this section), but for some reasonably-sized region, such a factor is sufficiently low to capture objects as thin as an accretion disc. If it's *still* not enough we can lower the factor even further.

But what if we want to find the point exactly at the *boundary* of R? Then, simply being *within* R is not enough. Well, it is not very computationally efficient to find a point exactly on the boundary; a better question to ask would be a point that is sufficiently close to it, which will give close enough of an approximation anyway. There's a clever way to utilise the machinery we developed in this chapter – simply set D to be the distance to the boundary of R, and let it be *negative* if $\mathbf{y}$ is within R. Then according to Eq. (5.8), h would become negative if the integrator is already within the region, and it would be forced to backtrack its solution until it gets within a desired distance, say 10^{-4}, to the boundary.

For the integration of the RTE Eq. (3.2), the story is the exact same. This time, the state vector that stores the values of the spectral radiance has some larger number of dimensions, say 81, and can be treated as a function of the geodesic parameter λ, i.e. $\mathbf{I}(\lambda)$. But the λ obtained directly from the integration of Eq. (1.7) is not suitable since we have integrated *backwards* in time – $\lambda = 0$ should be the end point for the integration of the RTE, not the start. We can resolve this by simply negating λ, and so the starting point of the integration is the negative of whatever parameter value the integrator has stopped at while solving for the geodesic. The initial value $\mathbf{I}_0$ will then be the spectral radiance of the thing that the integrator has hit – the background, some solid object, etc. If the background is empty then we can simply set $\mathbf{I}_0 = 0$.

Summary. Numerical integration using the RK4 method is used, along with adaptive step-size control via step-doubling and the Richardson extrapolation. The step size is also capped by proximity to a region of interest (objects within the scene) to prevent overshooting.

Chapter 6

Hermite Interpolation

There are many variables to use and consider when implementing the results of the fluid simulations into the renderer. These variables are not given as an analytic function in spacetime – they can't be, since they're numerically simulated. The best we can do is evaluate these functions over some kind of regular **grid**, and approximate the value of the function at some particular point by **interpolating** between these grid points.

Now, here we talk of a *regular* grid, but it doesn't have to be. The important requirement for the computational method we're going to use is that the set of grid points may be expressed as a **tensor product** of 1D arrays. It can't just be some collection of random points – we must be able to draw lines between these points that form hyper-rectangular cells – exactly what a grid is.

Note, also, that the grid doesn't have to correspond to anything physical. The variables don't even have to be coordinates in spacetime – the grid we're talking about is in the **parameter space** of the function we're approximating. If the function depends on N independent variables, then the parameter space is N-dimensional, and we would be talking of an N-dimensional grid that spans a region of the parameter space we're interested in.

There remains a question of how exactly we can perform the interpolation. There are many different kinds, but the one we employ is already in the name of this chapter.

Definition 6.1. Let $f(x)$ be a function on the unit interval $[0, 1]$. The cubic **Hermite interpolation** approximates the value of $f(x)$ for any $0 \leq x \leq 1$ as:

$$f(x) = f(0)H_{00}(x) + f(1)H_{01}(x) + f'(0)H_{10}(x) + f'(1)H_{11}(x), \tag{6.1}$$

where the various $H_{ij}(x)$'s are the Hermite **basis functions**:

$$\begin{cases} H_{00}(x) = 2u^3 - 3u^2 + 1, \\ H_{01}(x) = -2u^3 + 3u^2, \\ H_{10}(x) = u^3 - 2u^2 + u, \\ H_{11}(x) = u^3 - u^2. \end{cases} \tag{6.2}$$

Notice that Eq. (6.1) requires us to know not only the value of the function at the ends of the interval, but also that of its *derivative*. This can be separately calculated before performing the interpolation to maximise computational efficiency.

Definition 6.2. Let $f(x)$ be a function whose value at some x_-, x_0, and x_+ are given such that $x_- \leq x_0 \leq x_+$. The **finite difference method** states that:

$$f'(x_0) \approx \frac{f(x_+) - f(x_-)}{x_+ - x_-}, \tag{6.3}$$

essentially the slope of the secant line passing through the points $(x_-, f(x_-))$, and $(x_+, f(x_+))$.

The validity of Eq. (6.3) can immediately be seen by considering the definition of a derivative.

Now, Eq. (6.1) applies only to functions of a single variable, but more often than not we will be considering functions of *spacetime*, i.e. *four* variables. Luckily, we may easily extend this method to any number of dimensions iteratively.

Suppose, for instance, that we want to evaluate the function $f(x, y)$ at the point $(0.5, 0.5)$, but we only know its values (and that of its derivatives) at the corner of the unit square. We can apply Eq. (6.1) twice in the x-direction at both $y = 0$ and $y = 1$ to obtain an approximation for $f(0.5, 0)$ and $f(0.5, 1)$. But to get to $f(0.5, 0.5)$ with Eq. (6.1) we would need the value of $\partial f/\partial y$ at both $(0.5, 0)$ and $(0.5, 1)$ – to get those we would have to interpolate the *derivatives* in the previous step too. According to Eq. (6.1), we would need the derivative of $\partial f/\partial y$ in the direction we'd interpolated (the x-direction), so in addition to $\partial f/\partial x$ and $\partial f/\partial y$, the standard first-order derivatives of the function with respect to each of its variables, we would also need $\partial^2 f/\partial x \partial y$ evaluated at the corners of the unit square to complete the interpolation process.

In general, for a function of N independent variables, we would need to know its 2^N derivatives (for every variable pick if the function is to be differentiated with respect to it or not). For $N = 4$ that's 16 different values – we would need to store 16 copies of the same grid for each of the function's derivatives. In any case, the procedure described earlier works for any general N, and we would have to evaluate Eq. (6.1) a total of $(4^N - 1)/3$ times – that's 85 Hermite interpolations for a single query point.

We recognise that this is a lot of computational work, so we streamline the interpolation process further by rewriting Eq. (6.1) as

$$f(x) = ax^3 + bx^2 + cx + d, \qquad \begin{cases} a = 2f(0) - 2f(1) + f'(0) + f'(1), \\ b = -3f(0) + 3f(1) - 2f'(0) - f'(1), \\ c = f'(0), \\ d = f(0). \end{cases} \tag{6.4}$$

This way, one does not need to evaluate four separate polynomials Eq. (6.2) before multiplying and adding them up, and can instead simply compute the coefficients and evaluate one single polynomial Eq. (6.4). Even better, we can even evaluate the polynomial with **Horner's method**:

$$p(x) = a_0 + a_1 x + a_2 x^2 + \cdots = a_0 + x(a_1 + x(a_2 + x(\cdots))), \tag{6.5}$$

which has the advantage of lowering the computational cost even further.

So far, we've only considered interpolation within the unit hyper-cube, which suggests that grid points must be evenly spaced with length 1 – but it doesn't have to be this way. If the local grid defines a hyper-cube of dimension $h_1 \times h_2 \times \cdots h_N$, then we can simply scale it to match the unit hyper-cube, by dividing each coordinate x_i by h_i to obtain **normalised coordinates**, before applying the machinery we developed earlier. The upshot to this is that the derivatives must similarly be scaled by the chain rule, i.e. divided by the appropriate h_i's depending on which variables the function has been differentiated with respect to. This generalisation would allow interpolation to be done on any N-dimensional grid, regular or not.

Summary. Interpolation between grid points is done with the Hermite interpolation, which may be extended iteratively to any number of dimensions. The computational cost scales exponentially with the number of independent variables the function depends on. Also, all cross-derivatives of the function with respect to each of its variables must be given over the same grid, which may be approximated by the finite difference method. The Hermite interpolation may also be streamlined by rewriting Eq. (6.1) as a single polynomial and employing Horner's method.

Part III

Code

Chapter 7

Classes

In this part of the manual we explain the mechanism of the code. The code is **object-oriented**, i.e. it relies on the use of classes and their methods. This provides a neat way to organise the various components to the rendering engine. Note that as of now, not all of the features of Part I and II are implemented. The reader is also advised to consistently refer to the code from the GitHub repository since this section is particularly dense.

To accelerate the rendering process, we must **parallelise** the programme as much as we can. We use **Numba**, which can compile algorithms in Python and run them on speeds comparable to C and Fortran. Numba-compiled functions run in what is known as **nopython mode** – and to do so, simply decorate a function with the decorator `@njit` or a class `@jitclass(spec)` with **specifications** of each variable used within the class. The downside to this additional speed is that nopython mode greatly reduces the variety of techniques we may employ in the actual code.

To illustrate this, let's take a look at the class of three-vectors `Vec` (src/render/Classes/base.py):

```
1  spec_vec= [("x", float64), ("y", float64), ("z", float64)]
2  @jitclass(spec_vec)
3  class Vec:
4      def __init__(self, x, y, z):
5          self.x = x
6          self.y = y
7          self.z = z
8      def __add__(self, vec): return Vec(self.x+vec.x, self.y+vec.y,
          self.z+vec.z)
9      def __sub__(self, vec): return Vec(self.x-vec.x, self.y-vec.y,
          self.z-vec.z)
10     ...
11
12     def length(self): return np.sqrt(self.x**2 + self.y**2 + self.z
          **2)
13     def normal(self): ...
14     def dot(self, vec): ...
15     def cross(self, vec): ...
16     def np_array(self): return np.array([self.x, self.y, self.z])
17
18     def is_normal(self): ...
```

Here, in line 1, we can see that the specifications of the class `spec_vec` is a list of tuples, wherein the name of each property of the class (here `x`, `y`, and `z`) is followed with a variable *type*. In this case, the components of a vector are real floats, so we tell the computer to store these variables in

double-precision floating-point format, aka `float64`.

Next, the initialisation method `__init__` is specified – this method is called every time the class is initialised, say when we want to define a new vector. Most of the time it's just assigning the input variables to the property of the class, but we can add more features, such as checking if the inputted values are valid, or perform calculations that are useful to be computed beforehand so that it doesn't have to be rederived again and again.

Afterwards, **methods** for the class are defined. Here, `Vec` is a special case in that we would like to perform numerical operations on the class – so we define what 'addition', 'subtraction', 'multiplication', etc. through the methods `__add__`, `__sub__`, `__mul__`, etc. look like and tell the computer to return the corresponding values every time we add or subtract vectors together or multiply it with a scalar. Other important ones are `__eq__`, which defines if one vector is 'equal' to another, `__ne__`, which defines the opposite, and `__neg__`, which defines the negation of a vector.

Up until now it seems that methods are in general simply operations on a class – and that is indeed one way to think about it – but a better way is to think of them as *things you can do* with the object. Like return its magnitude (`length`), normalise it (`normal`), or return the numpy array corresponding to the vector (`np_array`). To actually call the methods of a class in practice, type `obj.method(args)`, where `obj` is a variable of type belonging to the class, `method` is the name of the method, and `args` are the arguments you put into the definition of the method (except for `self` – it's included in every method definition).

Now that the mechanism of a class is explained, it's time to get onto the others. Here's the class of bounding boxes named `BoundingBox` (src/render/Classes/base.py), which will be used later to describe the smallest axis-aligned 3D box that bounds a given shape.

```
spec_bb = [("min_corner", Vec.class_type.instance_type), ("
   max_corner", Vec.class_type.instance_type)]
@jitclass(spec_bb)
class BoundingBox:
    def __init__(self, min_corner:Vec, max_corner:Vec):
        self.min_corner = min_corner
        self.max_corner = max_corner
        if not ...:
            raise ValueError(...)

    def in_bb(self, pt:Vec) -> bool:
        ...
```

Here, you can see that previosuly Numba-jitted classes may be used as arguments in *another* Numba jitclass. In this case, both `min_corner` and `max_corner` are `Vec`'s, so we simply insert them into the specs. For custom classes like these Numba wants us to add a `.class_type.instance_type` suffix at the end, but treat this simply as a notational issue.

The class `BoundingBox` features input validation in its initialisation method. Since an axis-aligned box can be defined uniquely by the two ends of its diagonal, we can simply pick the points that have the smallest and largest coordinates. Now, if `min_corner` has a greater, say x-coordinate, than that of `max_corner`, then the logic will be messed up, so we ensure that is not the case by raising an error and telling the user to try again. The other method of the class `in_bb` just describe if a given `pt` is within the bounding box. Admittedly, this isn't a very useful class, but it's there as a rough measure of the size of objects in a scene.

Now, a major limitation with `@jitclass` is that they don't allow **subclasses**, classes that inherit properties and methods of its parent class. For example, we can't have specifically a class of normal vectors whose objects are also of type `Vec` and inherit all its methods, even though it would be very

useful. The solution is to, for each would-be subclass, give it some tag, and for each method specific to that subclass, call the method based on its tag. Here's how it works. Let's take a look at the class of shapes named `Shape` (src/render/Classes/classes.py):

```
spec_shape = [...]
@jitclass(spec_shape)
class Shape:
    def __init__(self, pos:Vec, rot:tuple[float,float,float], tag:int, radius:float=0, height:float=0, r_in:float=0, r_out:float=0):
        self.pos = pos
        self.rot = rot
        self.tag = tag
        ...

        if self.tag == SHAPE_NULL: ShapeNull.init(self)
        if self.tag == SHAPE_SPHERE: Sphere.init(self, radius)
        if self.tag == SHAPE_CYLINDER: Cylinder.init(self, radius, height)
        if self.tag == SHAPE_ANNULUS: Annulus.init(self, r_in, r_out, height)

    def closest_pt(self, pt:Vec) -> Vec:
        if self.tag == SHAPE_NULL: return ShapeNull.closest_pt(self, pt)
        if self.tag == SHAPE_SPHERE: return Sphere.closest_pt(self, pt)
        if self.tag == SHAPE_CYLINDER: return Cylinder.closest_pt(self, pt)
        if self.tag == SHAPE_ANNULUS: return Annulus.closest_pt(self, pt)

    def on_surface(self, pt:Vec) -> bool:
        ...

    def in_shape(self, pt:Vec) -> bool:
        ...

    def in_shape_int(self, x:Vec) -> float:
        ...

    def projection(self, pt:Vec) -> tuple[float, float]:
        ...
```

Now, any 3D shape we can think of under the sun has in general a position, and a rotation. Every `Shape` object must therefore specify the variables `pos` and `rot`, the latter of which are essentially the three **Euler angles**. We can use these angles to define a local coordinate axis (`X`, `Y`, and `Z`). But for each shape that exists, there are additional specific parameters, like `radius` for a sphere, `height` for a cylinder, or the inner and outer radii `r_in` and `r_out` for an annulus. In practice, when the class is called, they may or may not be specified depending on what kind of shape you want to call – that's why they are **default arguments** set at some null value. The appropriate and relevant properties of the object are only initialised in the `__init__` method depending on what kind of shape

is called – and the code *knows* that by taking a look at the `tag`, which is an integer value that's different for different shapes. Their actual values are defined in the file src/render/Classes/tags.py, but they aren't important – the point is, the code looks at the tag, and 'understands' what kind of shape to initialise. For every class method specific to the kind of shape, then, the appropriate method specific to that particular shape is used. Then, to simply call a cylinder, for example, type `cyl = Shape(pos=..., rot=..., tag=SHAPE_CYLINDER, ...)`.

This is illustrated best in the `closest_pt` method, which returns the point on the surface of the shape closest to a given `pt`. The algorithm is going to be different depending on what the shape is – it's pretty simple for a sphere but a lot more complicated for an annulus. We can call these different functions depending on the tag under the same class method. If the tag indicates that it's a sphere, call `Sphere.closest_pt`, which is an `@njit`-decorated function defined in another Python file, src/render/Classes/Shapes/sphere.py. Similarly if the tag indicates that it's a cylinder, call `Cylinder.closest_pt`, defined in src/render/Classes/Shapes/cylinder.py, and so on. We won't bother with the mathematical details here, the reader may go into the respective files and check that the algorithms are indeed correct.

The reader may also be wondering what `SHAPE_NULL` is – it is simply a placeholder for *nothing*. A scene with nothing in it contains a null shape (Numba doesn't like empty lists, and we will define a scene as a list of objects).

The other methods of this class are `on_surface`, which returns if a given `pt` is within 10^{-4} to the surface of the shape; `in_shape`, which returns if a given `pt` is *inside* the shape; `in_shape_int`, which returns the minimum distance to the surface of the shape, and is negative if `x` is within the shape (useful as discussed earlier in Chapter 5); and `projection`, which projects the surface of the shape onto the domain $[0, 1] \times [0, 1]$, which is useful for colouring. The projection is simply a spherical/toroidal projection depending on the topology of the shape.

The nice thing about coding classes like this, is that pretty much every other class that have different 'varieties' essentially follow the same logic, just with different names and methods. We even stored the methods specific to different tags in organised folders. For example, `ColField` (src/render/Classes/classes.py), which is a small class used entirely to colour the surface of an object. There are two types of colour fields that we implemented: a constant-colour field with tag `COLFIELD_CONSTCOL` and argument `col`, describing the colour as a vector whose entries are its RGB values, and a checkerboard pattern, with tag `COLFIELD_CHECKERBOARD` and arguments `col1` and `col2` that describe the two colours used. There is only one method in this class: `sample`, which takes in two coordinates `u` and `v` within the domain $[0, 1] \times [0, 1]$ and outputs a colour depending on the specified pattern.

The same goes for the class `Hittable` (src/render/Classes/classes.py), which is the type of object that goes into the scene (a scene can simply be defined as a list of hittables). It has two non-default arguments in addition to the tag: `shape`, a variable of the `Shape` class that encodes everything about the geometry of the object, and `col_field`, of the `ColField` class that encodes the colour (or *colours*) of the object. Right now it's still pretty boring – it only has one non-trivial type `HITTABLE_LIGHT` that simulates a Lambertian light-emitting object, and one method `color`, which returns the colour of the hittable at that specific point on the surface. For a light source that's just the colour emitted at that location.

Now, there's the more involved class of gravitational fields `GravField` (src/render/Classes/-classes.py). It's a misnomer – it more so describes a curvature of spacetime, and can be thought of as the background spacetime manifold in which the scene is rendered. It determines how the light moves and curves and all of the optical effects discussed earlier. There are four different types of metrics implemented: the Minkowski metric, the Schwarzschild metric (Definition 1.1), the Kerr metric (Definition 1.2), and the Kerr-Newman metric (Definition 1.3), and with them the parameters

of the black hole – the position of its centre `pos`, its mass `M`, its rotation axis `ax` (a normalised vector), its angular momentum `J`, and its charge `Q`. Note that a check on whether the resulting black hole is super-extremal is done by Eq. (1.6), and if so, an error is raised. Since Eq. (1.6) requires us to divide by M, the default value for `M` cannot be set to zero – we pick some small value like 10^{-15}.

The methods of `GravField` are:

- `sample_g`, that returns the value of the metric tensor evaluated at a spacetime point `x` as a 4×4 numpy array;
- `sample_Gamma`, that returns the value of the Christoffel symbols evaluated at a spacetime point `x` as a $4 \times 4 \times 4$ numpy array;
- `coord_pos`, that takes in a spacetime point `x` in Minkowski coordinates and outputs a numpy array describing the same point in metric coordinates;
- `mink_pos`, that essentially does the opposite of `coord_pos`;
- `jacobian`, that outputs the Jacobian matrix $\partial x_{\text{mtc}}/\partial x_{\text{mnk}}$ evaluated at a spacetime point `x` given in Minkowski coordinates;
- `jacobian_inv`, that outputs the *inverse* Jacobian matrix $\partial x_{\text{mnk}}/\partial x_{\text{mtc}}$ evaluated at a spacetime point `x` given in metric coordinates;
- `coord_vel`, that takes in a vector `v` at a spacetime point `x` both in Minkowski coordinates and outputs the numpy array describing the same vector `v` in metric coordinates;
- `mink_vel`, that essentially does the opposite of `coord_vel`;
- `null_cond`, that takes in a three-vector `V` in Cartesian coordinates at a spacetime point `x` and outputs the past-directed four-vector satisfying the null condition Eq. (1.9) whose spatial components correspond to `V`, in metric coordinates.

The reader may go into each of the Python files in src/render/Classes/GravFields and ensure that the expressions for the metric, Christoffel symbols, and coordinate transformations are all correct. In addition, the methods `coord_vel` and `mink_vel` simply take the Jacobian and inverse Jacobian matrices and multiply it with the inputted vector, and the procedure for the method `null_cond` is described in the last couple paragraphs of Chapter 1.

There is one last class that ties the entire thing together and captures all the information required to render a scene – it's the class `RenderSettings` (src/render/Classes/classes.py). It takes in a lot of arguments, the width `w` and height `h` of the image in pixels, the camera position `cam_pos` and direction it's facing `cam_dir`, the scene as a list of hittables `scene`, the background image `background` as a $w \times h \times 3$ numpy array that encodes the RGB color of every pixel in the image, the radius of the **background box** `bg_rad`, and the background gravitational field `grav_field`. The background box is essentially a sphere centred at the camera – think of it as something that the light rays can hit too, and the background image is mapped onto the inside surface of this sphere. Everything the camera can see, then, would be confined to the interior of this background box. Also, Numba dictates lists must contain objects of the same type, and be turned into a `numba.typed.List` object for class properties. That's why you can see we assign to the property `self.scene` the variable `List(scene)` and not simply `scene`.

Before diving into the mechanisms of the methods, it is important to understand how the camera works. We use the **pinhole camera model**, where the camera receives only the light rays that intersect its position (`cam_pos`). To determine the exact image the camera 'sees', we define a **viewport**, a plane perpendicular to `cam_dir` and a distance 1 away from `cam_pos`, whose width and height

are determined by the dimensions of the final image. Here, we make it so that it will always have a width of 2, and the length will be scaled according to the **aspect ratio**. If a light ray enters the camera and intersects the viewport at some pixel, that particular pixel will be coloured accordingly.

In `RenderSettings`, the method `ray_dir_px` method takes in some coordinate (`x`,`y`) on the image, and outputs the normalised vector pointing from `cam_pos` to the location corresponding to the *centre* of the pixel (`x`,`y`) on the viewport. This requires defining the viewport unit vectors `cam_u` and `cam_v`, calculated during initialisation, in such a way that they are orthonormal, and that the plane they define is perpendicular to the camera direction. Then, the viewport coordinates `u` and `v` are calculated, so that the vector pointing from `cam_pos` to the pixel is gotten simply by computing `cam_pos + u*cam_u + v*cam_v`. Normalising then gives the final result. The use of this is that we can launch geodesics in *that* specific direction, so that in reality light following that geodesic will come into the camera at that specific angle and colour the corresponding pixel.

The other methods are bit easier to understand. `sample_bg` is a method that takes in a polar and azimuthal coordinate `theta` and `phi` and returns the colour of the background image mapped onto the background box. The method `geodesic_eq` is precisely the right hand side of Eq. (5.4), and is the function that will allow us to perform numerical integration. The last method `escape` takes in the state vector `Y` used during numerical integration, and returns the corersponding distance to the camera position in physical space, minus the radius of the background box. The important thing about this is, if the function returns a value larger than 0, that means the geodesic the integrator is solving for has intersected the background box, and the integration may be terminated immediately.

Summary. Python classes in Numba are decorated with `@jitclass` encoding the variable type for each class property. Since subclasses are not allowed, we use tags that help differentiate between varieties of the same class, allowing variety-specific methods to be implemented. The camera is modelled with the pinhole camera model.